Standalone Terminal-Based Neural Machine Translation System

Course: Natural Language Processing & Applications (NLPA 2026) | **Module:** translate_cli.py | **Repo:** github.com/PanthDhoriyani/Machine-Translation-System

Author / Student Name:	*[Insert Name / Roll Number]*	**Framework:**	PyTorch 2.6.0+cu124, Transformers
Primary Script:	translate_cli.py (Standalone)	**Hardware Acceleration:**	NVIDIA CUDA GPU / CPU Auto-detect

1. Executive Summary & Objectives

Machine Translation (MT) automates the conversion of text between natural languages while preserving semantic nuance. While enterprise MT platforms incorporate distributed microservices and front-end web layers, lightweight local edge inference requires a self-contained, dependency-minimal interface. This project develops **translate_cli.py**, a standalone single-file CLI application leveraging pre-trained MarianMT sequence-to-sequence Transformer models. Key objectives achieved include: (1) zero-server operational overhead, (2) sub-second translation on CUDA GPU, (3) interactive multi-turn REPL mode with live pair switching, (4) one-shot CLI execution, and (5) automated BLEU metric verification.

2. System Architecture & Processing Pipeline

The system follows an encapsulated sequential pipeline:

- **Input Normalization:** HTML sanitization, space collapsing, and unicode NFC canonical decomposition.
- **Model Manager & Cache:** Dynamic mapping of language pairs to HuggingFace repository IDs with in-memory caching to eliminate redundant loads.
- **Subword Tokenization:** SentencePiece BPE encoding with truncation capped at 512 tokens.
- **Seq2Seq Generation:** Transformer Encoder-Decoder inference supporting configurable Beam Search (default=4) and Greedy search.
- **Detokenization & Reporting:** Output string decoding, latency timing in milliseconds, and optional sentence BLEU score calculation.

3. Model Catalogue & Supported Language Pairs

Source	Target	HuggingFace Model ID	Architecture	Params
English (en)	French (fr)	Helsinki-NLP/opus-mt-en-fr	6L-6L Transformer	74.4 M
English (en)	German (de)	Helsinki-NLP/opus-mt-en-de	6L-6L Transformer	74.4 M
English (en)	Spanish (es)	Helsinki-NLP/opus-mt-en-es	6L-6L Transformer	74.4 M
English (en)	Hindi (hi)	Helsinki-NLP/opus-mt-en-hi	6L-6L Transformer	74.4 M
French (fr)	English (en)	Helsinki-NLP/opus-mt-fr-en	6L-6L Transformer	74.4 M
German (de)	English (en)	Helsinki-NLP/opus-mt-de-en	6L-6L Transformer	74.4 M

4. Experimental Performance & Verification

Input Sentence	Target	Output Translation	Latency	Hardware
Hello, welcome to our machine translation system.	French (fr)	Bonjour, bienvenue dans notre système de traduction automatique.	896.2 ms	CUDA GPU
Good morning! How are you doing today?	German (de)	Guten Morgen, wie geht es dir heute?	891.1 ms	CUDA GPU
Subsequent session translations (cached)	French (fr)	Fast in-memory cache lookup	< 50 ms	CUDA GPU

5. Verification Screenshots & Demonstration Segments

Attach terminal screenshots into the respective segment frames below for project validation and grading:

Segment 1: CLI Command-Line Help Output (--help)

```
python translate_cli.py --help
```

■ [SUBMIT SCREENSHOT HERE: Segment 1]

Paste terminal screenshot here before final submission

Figure 1: Terminal display of command line argument parser with --src, --tgt, --beam, and --ref options.

Segment 2: One-Shot English to French Translation

```
python translate_cli.py "Hello, welcome to our machine translation system." --tgt fr
```

■ [SUBMIT SCREENSHOT HERE: Segment 2]

Paste terminal screenshot here before final submission

Figure 2: Execution output for English-to-French translation demonstrating model loading, generation, and 896ms latency.

Segment 3: One-Shot English to German Translation

```
python translate_cli.py "Good morning! How are you doing today?" --tgt de
```

■ [SUBMIT SCREENSHOT HERE: Segment 3]

Paste terminal screenshot here before final submission

Figure 3: Execution output for English-to-German translation utilizing Helsinki-NLP/opus-mt-en-de model.

Segment 4: Interactive REPL Session & Dynamic Pair Switching

```
python translate_cli.py --interactive
```

> **■ [SUBMIT SCREENSHOT HERE: Segment 4]**
>
> *Paste terminal screenshot here before final submission*

Figure 4: Multi-turn interactive terminal mode showing real-time translation, pair switching (:pair en de), and exit.

Segment 5: Reference-Based Quality Evaluation (BLEU Score)

```
python translate_cli.py "Hello world" --tgt fr --ref "Bonjour le monde"
```

> **■ [SUBMIT SCREENSHOT HERE: Segment 5]**
>
> *Paste terminal screenshot here before final submission*

Figure 5: Translation accuracy benchmark verifying hypothesis vs reference with BLEU = 100.0 / 100.

6. Conclusion & Summary

The standalone translate_cli.py implementation fulfills all functional translation and performance requirements specified in the NLPA 2026 project curriculum. By packaging state-of-the-art Transformer models into a single script with native GPU acceleration, memory caching, and dual-mode execution (CLI and REPL), it bridges the gap between complex research architectures and practical edge utility.